

Research project - Who adopts Coding Agents on GitHub

Michał Sobczak
University of Warsaw
Warsaw, Poland

ms440009@students.mimuw.edu.pl

ABSTRACT

Wraz z pojawieniem się agentów AI, takich jak Claude Code czy GitHub Copilot, proces wytwarzania oprogramowania ewoluował w stronę współpracy z autonomicznymi asystentami. Niniejsza praca analizuje zależność między poziomem doświadczenia programisty a sposobem wykorzystania dostępnych agentów. W badaniu wykorzystano zbiór danych **AIDev**, udostępniony w ramach konkursu **MSR 2026 Mining Challenge**. Dane obejmują aktywność ponad 72 000 użytkowników w ponad 116 000 publicznych repozytoriach w okresie od grudnia 2024 do sierpnia 2025, którzy łącznie utworzyli 932 791 *pull requestów*. Do klasyfikacji programistów wykorzystano zarówno dane z powyższego zbioru, jak i informacje pozyskane za pośrednictwem interfejsu **GitHub GraphQL API**. Użytkownicy zostali podzieleni na trzy grupy: **Junior**, **Mid** i **Senior**, w ramach których podjęto próbę odpowiedzi na następujące pytania badawcze:

- W jaki sposób doświadczenie programisty koreluje z częstotliwością wykorzystywania agentów AI?
- Czy doświadczenie człowieka przekłada się na wyższy współczynnik akceptacji kodu wygenerowanego przez agenta?
- Czy zmiany wprowadzone przez grupę juniorów przy użyciu AI wymagają większej liczby poprawek (mierzonej liczbą komentarzy) ze strony opiekuna projektu?

1 WSTĘP

Ewolucja narzędzi AI w programowaniu

W ostatnich latach rozwój sztucznej inteligencji znacząco przekształcił proces wytwarzania oprogramowania. Początkowo rola AI ograniczała się do prostych asystentów dostępnych w przeglądarkach, by następnie ewoluować w stronę integracji z IDE w formie systemów autouzupelniania kodu. Obecnie obserwujemy kolejny przełom - agenci tacy jak **Claude Code** zyskują dostęp do całych repozytoriów, co pozwala im nie tylko na naprawę błędów, ale i na samodzielne implementowanie nowych funkcjonalności.

Cele badawcze

Głównym celem niniejszej pracy jest zbadanie korelacji między poziomem doświadczenia programisty a sposobem wykorzystania narzędzi AI. W ramach analizy podjęto próbę zweryfikowania, czy programiści na początkowym etapie ścieżki zawodowej sięgają po wsparcie agentów AI częściej niż eksperci, oraz jak kształtuje się współczynnik akceptacji zmian w ramach *pull requestów* w zależności od stopnia zaawansowania użytkownika i obecności asystenta AI. Dodatkowo zbadano zależność mówiącą o tym, czy doświadczeni programiści potrafią na tyle precyzyjnie opisać zadanie agentowi, że generowany kod wymaga mniejszej liczby późniejszych poprawek.

Zbiór danych

Badania oparto na zbiorze danych **AIDev**, który jest obecnie największym i najbardziej aktualnym publicznie dostępnym zestawem danych tego typu. Kluczowym atutem tego zbioru jest fakt, że dotyczy on projektów komercyjnych, co znacząco podnosi wiarygodność wyników w kontekście rzeczywistego rynku pracy.

Podczas wstępnej analizy zauważyłem, że podział użytkowników na grupy Junior, Mid oraz Senior wyłącznie w oparciu o dane ze zbioru byłby niemiarodajny. Wnioskowanie o poziomie doświadczenia programisty jedynie na podstawie jego interakcji z agentami AI mogłoby prowadzić do błędnych wyników. Co więcej, liczba *pull requestów* utworzonych w tym zbiorze wyłącznie przez użytkowników jest relatywnie niewielka (6618 wierszy w zestawieniu z ogólną liczbą 932 791 rekordów), co uniemożliwiłoby rzetelne porównanie. W związku z tym, aby uzyskać pełniejszy obraz badanych osób, zdecydowałem się na pozyskanie dodatkowych informacji zewnętrznych.

GitHub GraphQL API

W celu wzbogacenia profilu użytkowników zdecydowałem się na pozyskanie dodatkowych metryk bezpośrednio z platformy GitHub. Ręczne sprawdzanie tak wielu profili przez przeglądarkę byłoby czasochłonne, dlatego zdecydowałem się na skorzystanie z API GitHuba. Początkowo testowałem standardowe REST API, ale szybko okazało się, że nie pozwala ono na wyciągnięcie wystarczająco szczegółowych informacji, a dodatkową barierą były restrykcyjne limity zapytań na godzinę. Po kilku eksperymentach, zrezygnowałem z tego podejścia na rzecz GitHub GraphQL API. Dzięki mniejszym limitom i szerszemu dostępowi do danych, udało mi się sprawnie uzyskać najważniejsze informacje o badanej grupie.

Dla każdego użytkownika pozyskałem szczegółowe dane obejmujące liczbę repozytoriów, w których brał on udział, liczbę publicznych i prywatnych *commitów* z ostatnich pięciu lat. Dodatkowo uwzględniłem statystyki dotyczące otwartych *pull requestów* oraz zgłoszonych *issues*. Ostatecznie udało mi się zgromadzić dane dla 70 486 osób z całej grupy 72 189. Na ich podstawie wytrenowałem las losowy oparty na drzewach decyzyjnych, aby przewidzieć brakujące statystyki. Wykorzystałem do tego model **Random Forest Regressor**, który przewiduje ciągłe wartości dla brakujących użytkowników. Liczby *commitów* czy zrobionych *review* to liczby całkowite - wynik jest zaokrąglany do liczb całkowitych.

Warto zaznaczyć, że walidacja modelu (podział 80/20) wykazała ograniczoną moc predykcyjną cech ze zbioru AIDev względem ogólnej aktywności użytkownika na GitHubie (R^2 w granicach 0.05–0.15, a w części przypadków ujemny), co sugeruje, że oba źródła danych opisują częściowo odmienne aspekty aktywności, a wartości uzupełnione dla brakujących użytkowników należy traktować z pewną ostrożnością.

2 METODOLOGIA

Dla każdego użytkownika wyznaczyłem zestaw parametrów opisujących jego aktywność w repozytorium GitHub

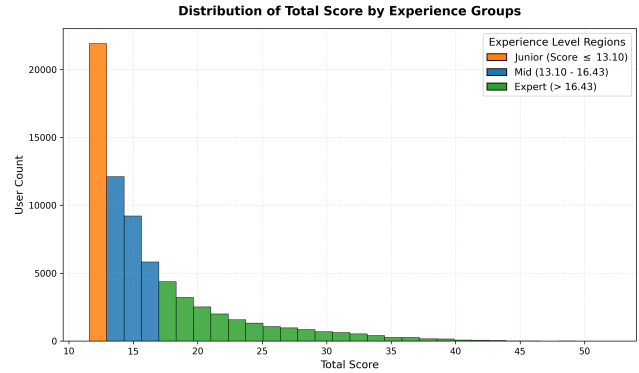
- Liczba projektów, w których użytkownik był zaangażowany jako współtwórca.
- Liczba dokonanych *commitów* w repozytoriach publicznych w ciągu ostatnich 5 lat.
- Liczba dokonanych *commitów* w repozytoriach prywatnych w ciągu ostatnich 5 lat.
- Liczba przeprowadzonych *code review* w ciągu ostatnich 5 lat.
- Liczba własnych repozytoriów utworzonych w ciągu ostatnich 5 lat.
- Liczba *followersów*.
- Maksymalna liczba *stars* zdobytych przez pojedyncze, własne repozytorium.
- Maksymalna liczba *forks* pojedynczego, własnego repozytorium.
- Staż konta na platformie GitHub (czas od rejestracji).
- Liczba utworzonych *pull requestów* zarejestrowana w zbiorze **AIDev**.
- Całkowita liczba *commitów* odnotowana w zbiorze **AIDev**.
- Liczba przeprowadzonych recenzji kodu na podstawie danych ze zbioru **AIDev**.
- Liczba zgłoszonych *issues* pochodząca ze zbioru **AIDev**.

2.1 Podejście wstępne

Początkowe podejście klasyfikacyjne opierało się na niezależnej analizie każdego z parametru za pomocą prostego systemu punktowego. Dla każdej z badanych cech wyznaczono medianę oraz 90. percentyl, które posłużyły jako punkty odniesienia w procesie punktacji. Następnie zastosowano funkcję przypisującą wyniki w skali od 0 do 4 punktów zadanej następującym wzorem:

$$f(x) = \min(4, \max(0, y(P_{90}) + \frac{y(P_{90}) - y(P_{50})}{P_{90} - P_{50}}(x - P_{50})))$$

gdzie P oznacza funkcję centylową (czyli P_{50} to mediana), oraz $y(P_{50}) = 1$ oraz $y(P_{90}) = 3$. Ostateczny wynik dla użytkownika stanowił kombinację liniową punktów uzyskanych we wszystkich rozpatrywanych kategoriach. Na podstawie tak obliczonej wartości sumarycznej dokonano ostatecznego podziału użytkowników na trzy grupy doświadczenia: Junior, Mid oraz Senior.



Liniowy model punktowy oparty na percentylach

Metoda ta wykazała jednak istotne wady metodologiczne. Przede wszystkim traktowała każdy parametr jako całkowicie niezależny, ignorując istniejące korelacje. Ponadto model ten charakteryzował się wysoką arbitralnością - progi centylowe oraz finalny podział na trzy grupy opierały się na subiektywnie dobranych wartościach, co uniemożliwiało rzetelną weryfikację statystyczną.

2.2 Podejście docelowe

W finalnym etapie analizy zastosowano nienadzorowane podejście statystyczne w postaci algorytmu *k-means*. Analizowany zbiór składa się wyłącznie ze zmiennych ilościowych charakteryzujących się znaczną rozpiętością wartości (np. liczba publicznych repozytoriów waha się w przedziale od 1 do 617 754). Aby zniwelować ten efekt i zapewnić równy wpływ wszystkich zmiennych na proces grupowania, dane poddano transformacji kwantylowej, sprowadzającej rozkłady zmiennych do zbliżonych do normalnego, a standaryzacji cech według formuły:

$$z = \frac{x - \mu}{\sigma}$$

gdzie μ oznacza średnią arytmetyczną, a σ odchylenie standardowe danej cechy.

W celu zwiększenia stabilności i wiarygodności wyników, ze zbioru usunięto obserwacje odstające. Wykorzystano do tego algorytm **Isolation Forest** - wydajny algorytm służący do szukania anomalii, z parametrem zanieczyszczenia ustalonym na poziomie 1%. Następnie, w celu redukcji szumu oraz umożliwienia czytelnej wizualizacji wielowymiarowych przestrzeni, zastosowano analizę głównych składowych (**PCA**). Ograniczenie wymiarowości do dwóch głównych komponentów pozwoliło na zachowanie kluczowych struktur danych przy jednoczesnym odrzuceniu 5% wariacji, traktowanej jako szum informacyjny. Tak przygotowane dane posłużyły jako wejście do algorytmu *k-means*.

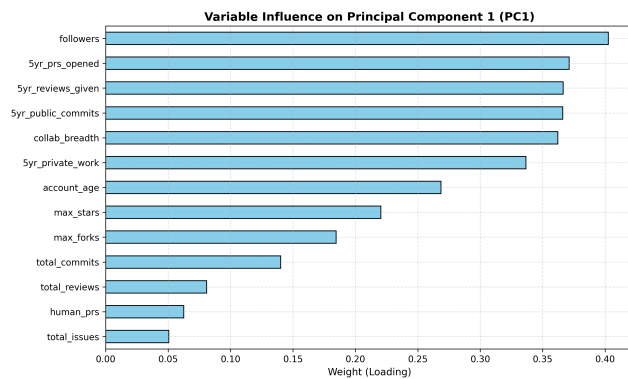


Figure 1: PCA PC1

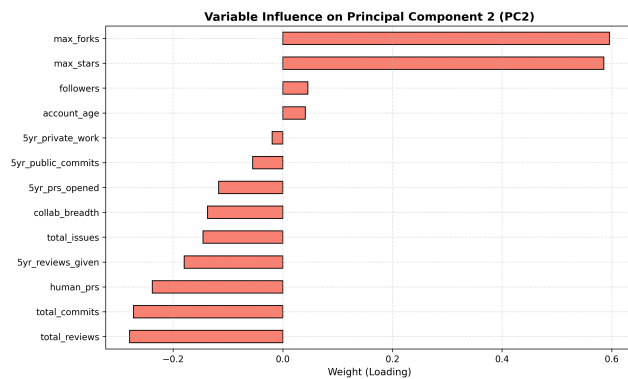
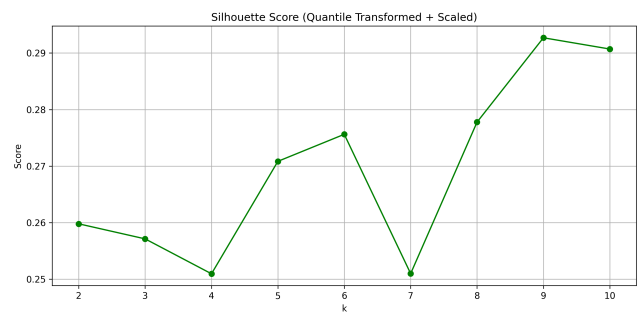


Figure 2: PCA PC2

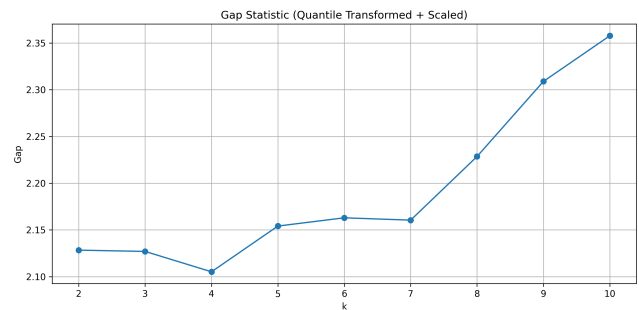
2.3 Jak odpowiednio dobrać parametr k?

Do wybrania optymalnej liczby klastrów użyłem dwóch metod:

- **Gap Statistic:** Metoda oceny jakości klastrowania, która porównuje całkowitą wariancję wewnątrz klastrową danych rzeczywistych z oczekiwaną wariancją dla danych referencyjnych wygenerowanych z rozkładu jednostajnego.
- **Silhouette Method:** Mierzy, jak dobrze każdy punkt pasuje do swojego klastra w porównaniu do klastrów sąsiednich. Wynik ten oscyluje między 1 a -1, gdzie wyższe wartości oznaczają lepiej odseparowane i gęstsze klastry.



Silhouette Method



Gap statistic

Ostatecznie wybrałem wartość $k = 3$. Przy $k = 4$ zauważalny jest spadek jakości podziału, a dobór niskiej wartości parametru zapobiega nadmiernemu rozdrobnieniu danych i tworzeniu trudnych w interpretacji podgrup. Dodatkowo w świecie biznesu najczęstszy jest taki podział właśnie, a mianowicie na: Juniorów, Midów oraz Seniorów.

Warto zauważyć, że algorytm *k-means* realizuje grupowanie w sposób nienadzorowany - wyznacza skupienia, ale nie nadaje im konkretnych etykiet. Aby nadać grupom znaczenie i przypisać im odpowiednie nazwy, wykorzystano analizę głównych składowych. Pozwoliła ona wyznaczyć atrybuty mające największy wpływ na zróżnicowanie danych. Na tej podstawie zinterpretowano profile poszczególnych klastrów:

- **Juniorzy:** grupa użytkowników, którzy w kluczowych zmiennych osiągnęli zerowe bądź minimalne wyniki.
- **Seniorzy:** grupa skupiająca osoby o najwyższych wartościach w najważniejszych parametrach.
- **Mid:** grupa pośrednia, charakteryzująca się umiarkowanymi wynikami w analizowanych metrykach.

3 WYNIKI

3.1 W jaki sposób doświadczenie programisty koreluje z częstotliwością wykorzystywania agentów AI?

W celu oceny zależności między poziomem doświadczenia programistów a częstotliwością wykorzystania agentów AI przeprowadzono nieparametryczny test Kruskala-Wallisa oraz testy post-hoc Manna-Whitneya z korektą Bonferroniego.

Test Kruskala-Wallisa wykazał istotną statystycznie różnicę w częstotliwości generowania *pull requestów* między analizowanymi grupami doświadczenia.

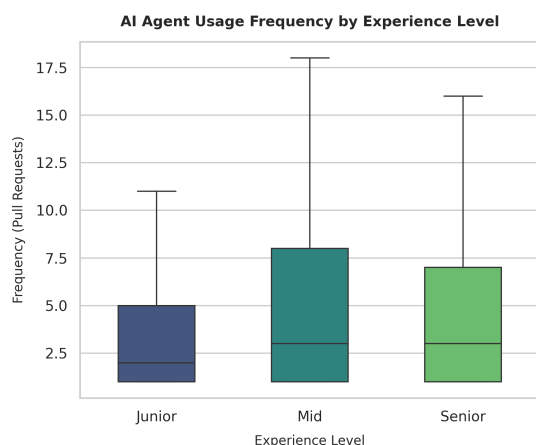
Analiza post-hoc wykazała następujące zależności:

- Grupa **Junior** charakteryzuje się istotnie niższą częstotliwością używania agentów AI w porównaniu z pozostałymi grupami.
- Między grupami **Mid** a **Senior** nie odnotowano istotnych różnic statystycznych.

Najwięcej z agentów AI korzystają programiści na poziomie **Mid** oraz **Senior**, osiągając przy tym najwyższą medianę wykorzystania.

Summary statistics

Level	N	Mean	SD	Mdn
Junior	53,327	9.25	47.97	2.0
Mid	16,192	21.94	407.51	3.0
Senior	1,948	17.76	97.93	3.0



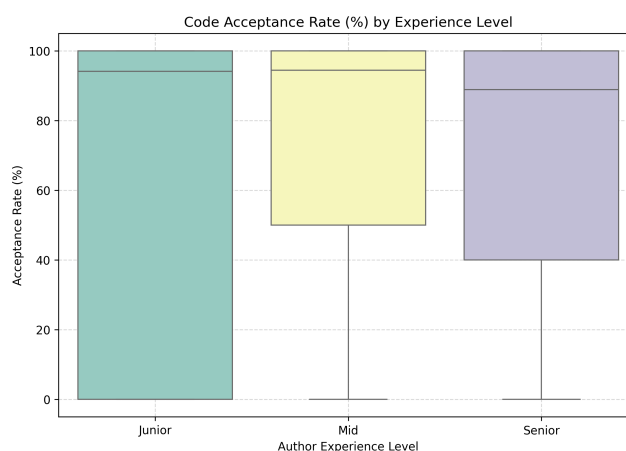
3.2 Czy doświadczenie człowieka przekłada się na wyższy współczynnik akceptacji kodu wygenerowanego przez Agentą?

Przeprowadzona analiza wykazuje, że poziom doświadczenia programisty wpływa na współczynnik akceptacji kodu wygenerowanego

przez agentów AI. Test Kruskala-Wallisa potwierdził istnienie istotnych różnic między badanymi grupami. Szczegółowa analiza testami post-hoc (Mann-Whitney U z korektą Bonferroniego) ujawnia następujące zależności:

- Programiści na poziomie **Mid** osiągają najwyższą średnią akceptowalności kodu oraz najwyższą medianę.
- Kod generowany przez **Seniorów** oznacza się niższą medianą akceptacji wśród pozostałych grup, lecz testy nie wykazały istotnych różnic między pozostałymi grupami ($p = 0.43$ w przypadku **Juniorów** oraz $p = 1.00$ w przypadku **Midów**)

Na podstawie danych można wywnioskować, że doświadczenie człowieka nie przekłada się automatycznie na wyższy wskaźnik akceptacji kodu. Największą efektywność w przyjmowaniu propozycji agentów AI wykazują programiści na poziomie **Mid**.



Summary statistics

Level	N	Mean	SD	Mdn
Junior	51,554	63.33	44.05	94.12
Mid	12,857	71.55	38.31	94.44
Senior	1,945	67.44	40.03	88.89

3.3 Czy zmiany wprowadzone przez grupę juniorów przy użyciu AI wymagają większej liczby poprawek (mierzonej liczbą komentarzy) ze strony opiekuna projektu?

Analiza opierała się na zliczaniu komentarzy pozostawionych przez opiekunów, wewnątrz *pull requestów*. W przypadku gdy autorem

pull requesta okazywał się bot, sprawdzane było powiązane z nim *issue* i to jego twórcę traktowany był jako rzeczywistego autor zmian.

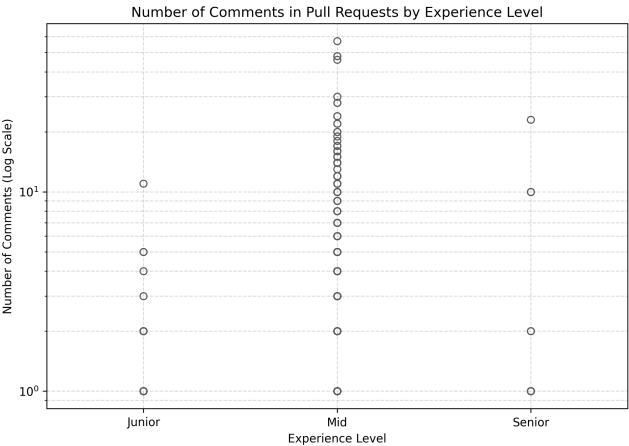
Do zweryfikowania tej hipotezy użyto testu **Kruskala - Wal-lisa** (odpornego na dużą liczbę wartości zerowych w danych). Test potwierdził istnienie statystycznych różnic pomiędzy wszystkimi grupami. Analiza testami Manna - Whitneya z korektą Bonferoniego wykazała, że różnice są istotne na każdym poziomie zestawień:

- **Junior vs Mid:** $p < 0.001$
- **Junior vs Senior:** $p < 0.001$
- **Mid vs Senior** $p < 0.001$

Kod generowany lub wspierany przez AI w przypadku Juniorów wygenerował najniższą średnią liczbę komentarzy. Najwyższy wskaźnik odnotowano w grupie **Mid**. Wynik ten jest w pewnym stopniu zaskakujący w kontekście postawionej hipotezy i może wynikać z tego, że Juniorzy tworzą relatywnie prostsze *pull requesty*, podczas gdy Mid-owie podejmują się bardziej złożonych zmian generujących więcej uwag ze strony maintainerów - niekoniecznie z powodu niższej jakości kodu.

Summary statistics

Level	N	Mean	SD	Mdn
Junior	490,767	0.00014	0.02288	0.0
Mid	305,409	0.00701	0.29589	0.0
Senior	34,607	0.00170	0.14711	0.0



ACKNOWLEDGMENTS

Praca ta powstała w ramach projektu badawczego 2025/2026 pod nadzorem prof. Krzysztofa Stencła.